Университет ИТМО
Факультет программной инженерии и компьютерной техники

---

Группа: P3311
Студент: Горошников Т.И.
Преподаватель: Потемин И.С.

# Отчет по лабораторной работе №3 “Формирование распределения случайных величин в заданной области определения“

## Цель работы

Изучить методы формирования заданных распределений случайных координат и направлений лучей в пространстве. Доказать корректность полученных распределений случайных величин.

## Доказательство корректности используемых методов

Треугольник

Пусть $P_0$, $P_1$ и $P_2$ — вершины треугольника. Введем систему координат $(U, V)$, где $U$ и $V$ — смещения по сторонам треугольника $P_0P_1$ и $P_0P_2$ соответственно.
Пусть $\xi$ — величина, равномерно распределенная по осям $U$ и $V$.
Следовательно, плотность распределения точек в треугольнике будет пропорциональна элементарным площадям $dS = \frac{du \cdot dv \cdot \sin\alpha}{2}$.
Данная величина постоянна по всей поверхности треугольника, поэтому координаты точки будут равномерно распределены, если будут вычислены как:

$$P = U_r\xi_u + V_r\xi_v$$

где $\xi_u$ и $\xi_v$ — величины, равномерно распределенные в диапазоне $[0; 1]$;
$U_r$ и $V_r$ — длины сторон треугольника.
Итоговая формула для координат точки:

$$P = P_0 + U_r\xi_u + V_r\xi_v$$

Круг

Пусть $O$ — центр круга, $R$ — радиус круга.
Рассмотрим описанный квадрат со стороной $2R$. Плотность распределения точек внутри него будет пропорциональна элементарным площадям $dS = dx \cdot dy$, где $dx$ и $dy$ — смещения по двум смежным сторонам квадрата. Следовательно, координаты точки в квадрате будут равномерно распределены при расчете по следующей формуле:

$$P = R(\xi_x + \xi_y)$$

где $\xi_x$ и $\xi_y$ — величины, равномерно распределенные в диапазоне $[0; 1]$.
NB: в программной реализации центр системы координат совпадает с центром круга, соответственно, $\xi_u$ и $\xi_v$ распределены в диапазоне $[-1; 1]$.
После генерации координат точки, необходимо провести проверку — попадает ли она в круг:

$$x^2 + y^2 \leq R^2$$

При промахе точка генерируется заново.
Итоговая формула для координат точки:

$$P = O + R\xi_x + R\xi_y$$

Сфера

Пусть $O$ — центр сферы, $R_S$ — радиус сферы.
Координаты точки на поверхности сферы имеют вид:

$$(R_S \sin\theta\cos\varphi; R_S \sin\theta\sin\varphi; R_S\cos\theta)$$

Для формирования равномерного распределения точек на поверхности сферы в сферических координатах необходимо построить функцию плотности вероятности, соответствующую площади сферического сегмента с фиксированным приращениям координат $d\varphi$ и $d\theta$:

$$dS = d\varphi \cdot d\theta \cdot \sin\theta$$

Плотность вероятности постоянна для $\varphi$, но зависит от $\sin\theta$ для $\theta$. Следовательно, необходимо нормировать плотность вероятности:

$$\int\limits_0^\pi C\sin\theta d\theta = 1 \Rightarrow C(-\cos\theta)|_0^\pi = 1 \Rightarrow 2C = 1 \Rightarrow C = \frac{1}{2}$$

Тогда функция распределения вероятности будет иметь вид:

$$P(\theta) = \frac{1}{2}\int\limits_0^\theta \sin t\, dt = \frac{1}{2}(1-\cos\theta)$$

Обратная функция распределения вероятности будет иметь вид:

$$P^{-1}(\theta') = \theta = \arccos(2\theta' - 1)$$

где $\theta'$ — приведенный обратный угол, равномерно распределенный в диапазоне $[0;1]$.
Следовательно, для построения равномерного распределения точек на поверхности сферы необходимо формирование пары чисел:

$$\varphi = 2\pi\xi_\varphi,\ \theta = \arccos(2\xi_\theta - 1)$$

где $\xi_\varphi$ и $\xi_\theta$ — величины, равномерно распределенные в диапазоне $[0;1]$.
После этого эти числа используются для вычисления координат. Итоговая формула для координат точки:

$$P = O + (R_S \sin\theta\cos\varphi; R_S \sin\theta\sin\varphi; R_S\cos\theta)$$

Косинусное распределение

Рассмотрим элементарный телесный угол:

$$dw = \sin\theta \cdot d\theta \cdot d\varphi$$

Косинусное распределение плотности направлений в сферической системе координат будет пропорционально $\cos\theta \sin\theta$.
Аналогично с плотностью точек на сфере, нормируем $\cos\theta \sin\theta d\theta$. Получаем:

$$p(\theta) = 2\cos\theta \sin\theta$$

Интегрируем функцию плотности распределения, находим функцию распределения вероятности:

$$P(\theta) = \sin^2\theta$$

Вместо обращения функции распределения вероятности, рассмотрим следующее преобразование. Пусть $\xi$ — величина, равномерно распределенная в диапазоне $[0; 1]$:

$$\xi = \sin^2\theta$$

$$\sin\theta = \sqrt{\xi},\ \ \cos\theta = \sqrt{1-\xi}$$

Следовательно, для нахождения косинусно распределенных направлений относительно нормали, необходимо найти следующие величины:

$$\varphi = 2\pi\xi_\varphi,\ \ \sin\theta = \sqrt{\xi_\theta},\ \ \cos\theta = \sqrt{1-\xi_\theta}$$

И вычислить сферические координаты вектора с их помощью.

### Программа для генерации случайных величин

Весь код можно также найти по ссылке.
Фрагмент файла `main.py` с основной логикой генерации:

```
import numpy as np

import matplotlib.pyplot as plt




rng = np.random.default_rng(35)
```

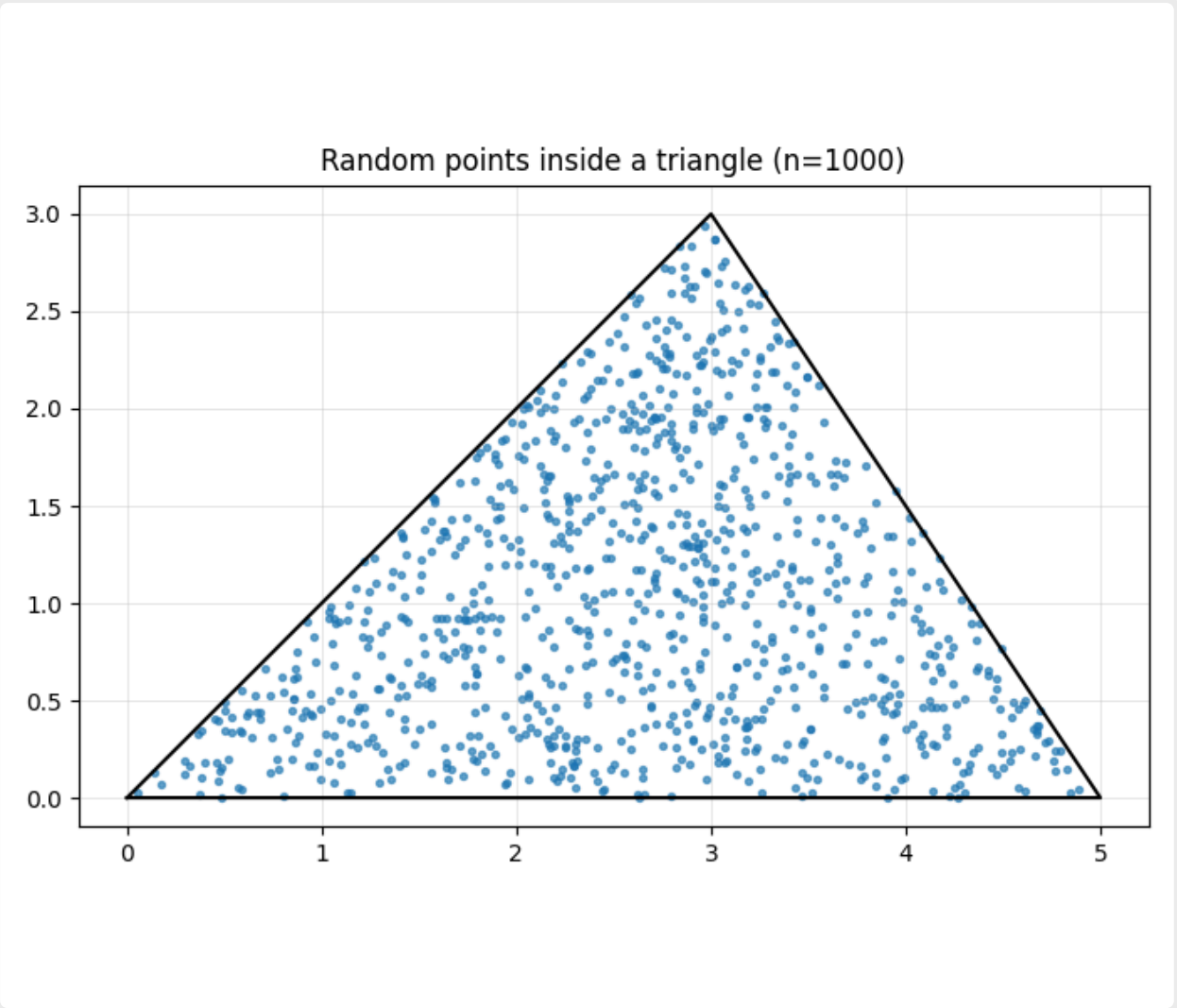
```
def generate_random_triangle_point(p0, p1, p2):

    u, v = p1 - p0, p2 - p0

    e_u, e_v = rng.uniform(), rng.uniform()

    if (e_u + e_v > 1):

        e_u = 1 - e_u

        e_v = 1 - e_v

    return p0 + u * e_u + v * e_v



def generate_random_circle_point(n, r, c):

    n = n / np.linalg.norm(n)



    a = np.array([0.0, 0.0, 1.0]) if abs(n[2]) < 0.999 else
np.array([1.0, 0.0, 0.0])

    u = np.cross(a, n)

    u = u / np.linalg.norm(u)

    v = np.cross(n, u)



    while True:

        e_u, e_v = rng.uniform(-1.0, 1.0), rng.uniform(-1.0, 1.0)

        if (e_u ** 2 + e_v ** 2 <= 1.0):

            break
```

```
    return c + r * (u * e_u + v * e_v)



def generate_random_sphere_point(r, c):

    e_phi, e_theta = rng.uniform(), rng.uniform()

    phi = 2 * np.pi * e_phi

    theta = np.acos(2 * e_theta - 1)

    return c + np.array([r * np.sin(theta) * np.cos(phi), r *
np.sin(theta) * np.sin(phi), r * np.cos(theta)])



def generate_random_cosine_direction(n):

    n = n / np.linalg.norm(n)



    a = np.array([0.0, 0.0, 1.0]) if abs(n[2]) < 0.999 else
np.array([1.0, 0.0, 0.0])

    u = np.cross(a, n)

    u = u / np.linalg.norm(u)

    v = np.cross(n, u)



    e_phi, e_theta = rng.uniform(), rng.uniform()

    phi = 2.0 * np.pi * e_phi
```

```
    sin_theta = np.sqrt(e_theta)

    cos_theta = np.sqrt(1.0 - e_theta)



    d = u * (np.cos(phi) * sin_theta) + v * (np.sin(phi) * sin_theta) +
n * cos_theta

    return d / np.linalg.norm(d)
```

## Результаты работы программы

Треугольник

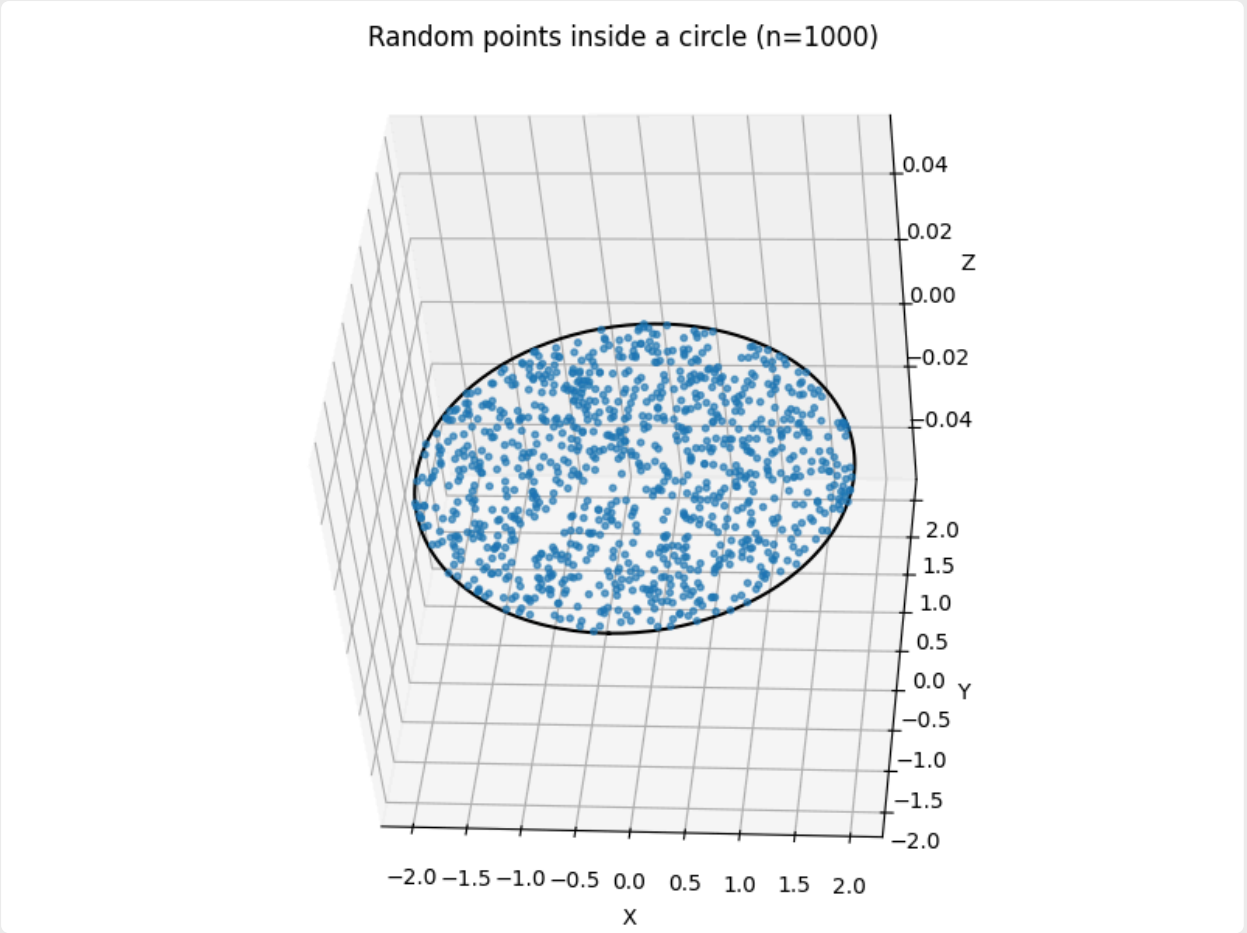



Случайно сгенерированные точки внутри треугольника

Для проверки корректности работы программы, треугольник разбивается на три равновеликих треугольника и в каждом из них считается количество точек. Результат для $n = 10000$:

```
Counts in regions: [3256 3405 3339]
Expected per region: 3333.33
Absolute deviation: [77.33 71.67  5.67]
Relative deviation (%): [2.32 2.15 0.17]
```

Круг

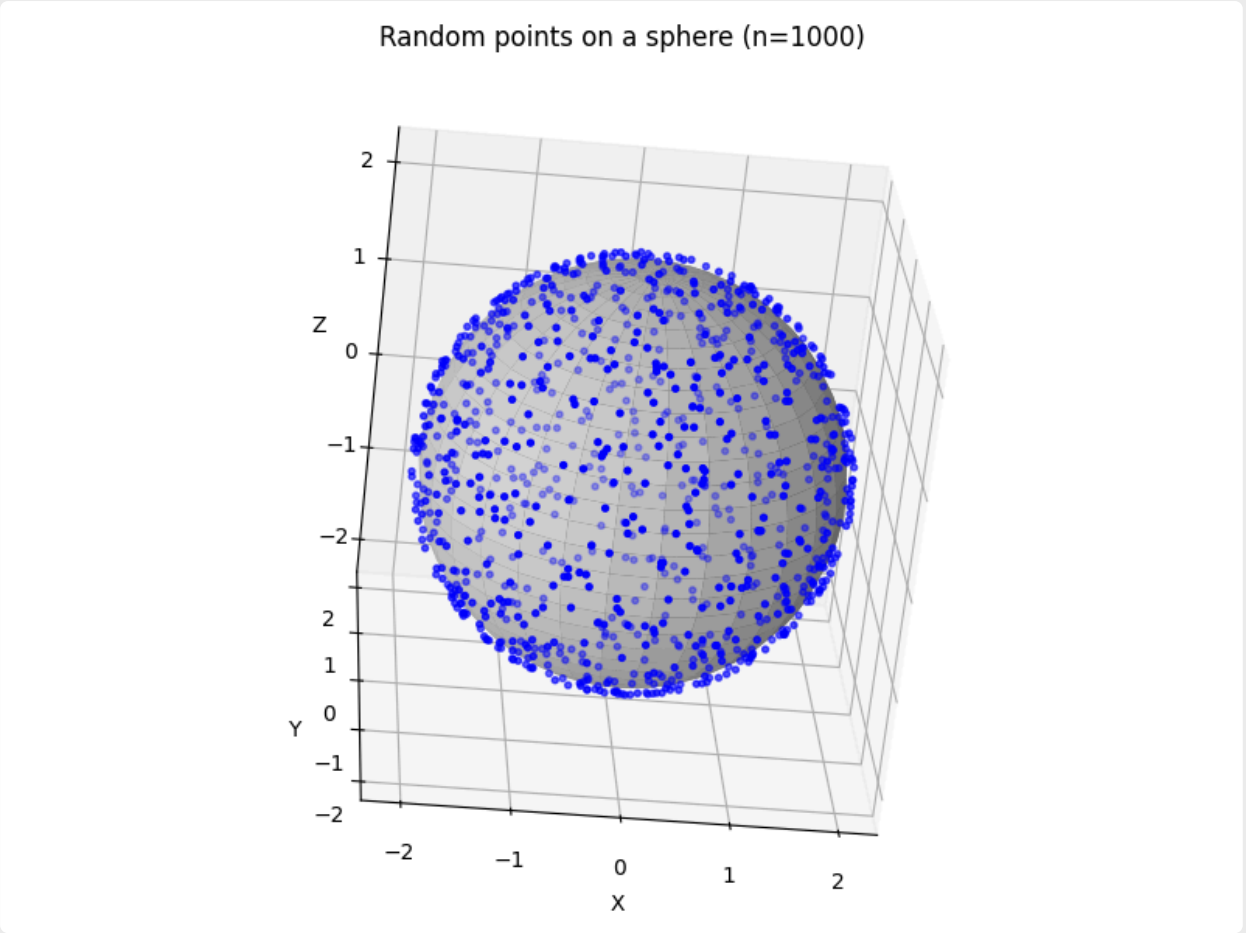



Случайно сгенерированные точки внутри круга

Для проверки корректности работы программы, круг разбивается на три равновеликих сектора и в каждом из них считается количество точек. Результат для $n = 10000$:

```
Counts in regions: [3362 3370 3268]
Expected per region: 3333.33
Absolute deviation: [28.67 36.67 65.33]
Relative deviation (%): [0.86 1.1  1.96]
```

## Сфера

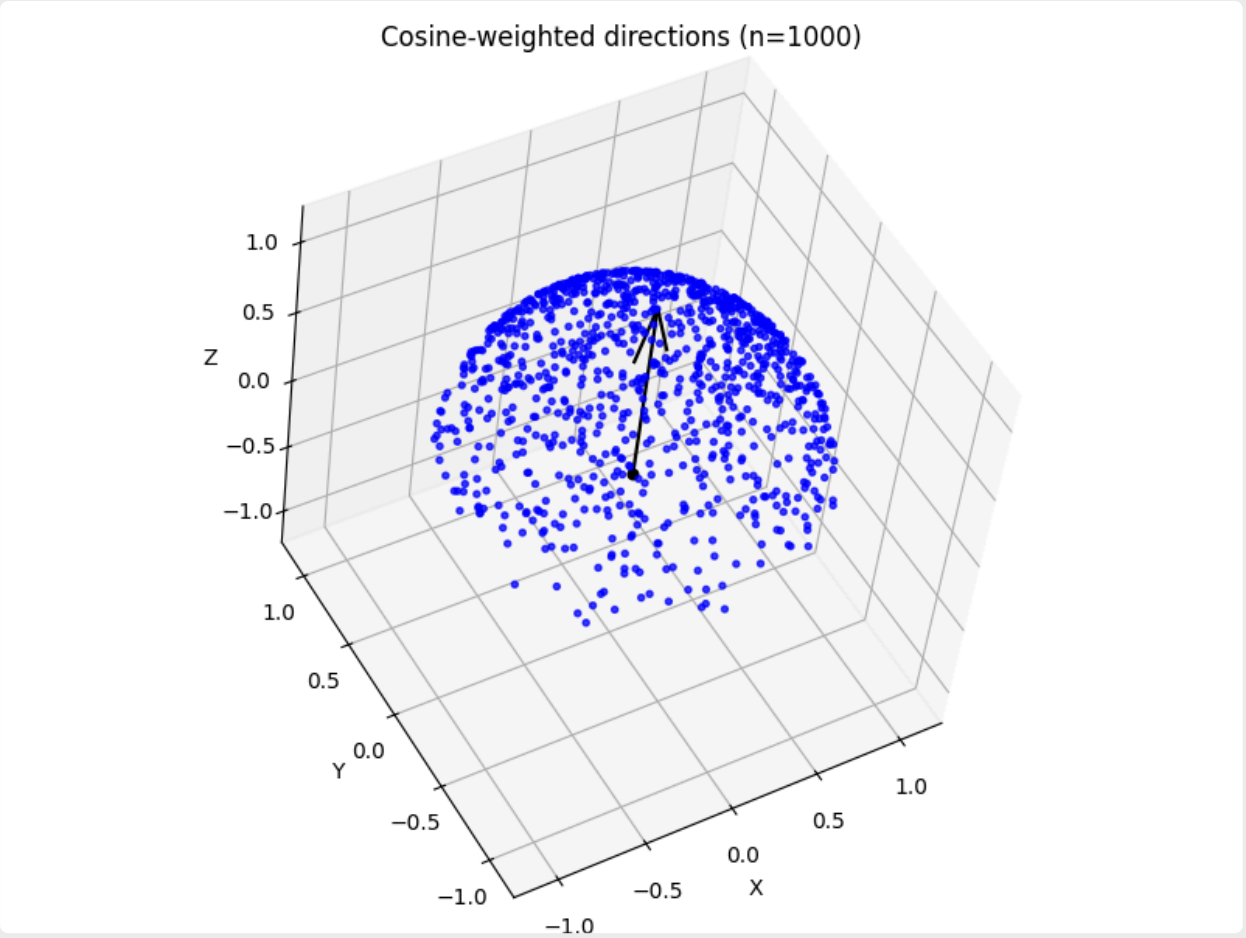



Случайно сгенерированные точки на сфере

Для проверки корректности работы программы, на сфере выбираются три не пересекающихся равновеликих сферических сегмента и в каждом из них считается количество точек.
Результат для $n = 10000$:

```
Counts in regions: [696 638 657]
Expected per region: 669.87
Absolute deviation: [26.13 31.87 12.87]
Relative deviation (%): [3.9  4.76 1.92]
```

Косинусное распределение




Случайно сгенерированные косинусно распределенные направления

Для проверки корректности работы программы, на полусфере выбираются три не пересекающихся равновеликих сферических сегмента и в каждом из них считается количество точек. Сегменты выбираются не случайно: один сегмент выбирается ближе к концу нормали, другой ближе к нижней части полусферы, а третий между ними. Производится оценка, сколько точек должно попасть в каждый регион, учитывая, что распределение косинусное.
Результат для $n = 10000$:

```
Counts in regions: [676 479 182]
Expected per region: [647.03 473.03 173.35]
Absolute deviation: [28.97  5.97  8.65]
Relative deviation (%): [4.48 1.26 4.99]
```

## Вывод

В ходе данной лабораторной работы я научился генерировать случайные точки и направления в двухмерном и трехмерном пространствах.